

Proyecto 1: ÁvilaOS

Diseño de un Sistema Operativo Distribuido

Asignatura: Sistemas Operativos

Trimestre: 2627-1

Profesor: García

Integrantes:

- Stefano Di Michelangelo — V-29678557
- Jesús Bethencourt — V-30906782
- Jesús Fernández — V-29965948

Repositorio: https://github.com/Stefano-dmm/AvilaOs---Sis.-Op

Caracas, octubre de 2026

1. Introducción

ÁvilaOS es un simulador de un sistema operativo distribuido sencillo. Modela un clúster de dos a ocho computadores; cada uno tiene su propia CPU, su memoria principal (RAM) y su propio núcleo con planificador y colas. Un reloj global marca el ritmo: en cada ciclo, cada computador avanza exactamente un ciclo de ejecución. Los procesos tienen un único hilo y pueden ser de cuatro tipos (CPU bound, I/O bound, productor y consumidor); los productores y consumidores se comunican con buffers de capacidad limitada sincronizados con semáforos, con acceso local o remoto (con latencia de red).

El proyecto está hecho en Java 21 con NetBeans, con interfaz gráfica en Swing, sin librerías externas y sin colecciones del framework de Java (todas las estructuras de datos son propias).

2. Arquitectura

El sistema está organizado en capas; cada una conoce solo a la de abajo, y la interfaz gráfica solo observa al sistema.

Capa	Paquete	Responsabilidad
Hardware	`avilaos.hardware`	`RelojGlobal`, `Clockable`, `CPU`, `RAM`: componentes que obedecen y no deciden
Sistema operativo de un computador	`avilaos.nucleo`	`Computador`: admisión, planificador, colas, bloqueos, deadlines
Sincronización	`avilaos.sincronizacion`	`SemaforoSO`, `BufferSO`: semáforos propios y buffer acotado
Procesos	`avilaos.modelo`, `avilaos.nucleo.programas`	`Proceso` (PCB), enums y el programa que ejecuta cada tipo de proceso
Distribución	`avilaos.nucleo`	`Cluster`: computadores, buffers, latencia de red, balanceo y reloj
Observadores	`avilaos.gui`, `avilaos.metricas`, `avilaos.config`	Ventanas, métricas y archivos de configuración
Estructuras	`avilaos.estructuras`	`Nodo`, `ListaEnlazada`, `Cola`, `Pila` propias

Hilos y exclusión mutua. El reloj global es un hilo; cada computador corre además en su propio hilo. En cada ciclo el reloj avisa a todos los computadores y espera a que todos terminen antes de pasar al siguiente (una barrera hecha con `Semaphore` de Java). Mientras un ciclo se ejecuta, el reloj mantiene tomado un cerrojo (un `Semaphore`) que la interfaz también toma para leer o modificar el estado; así nunca se ve ni se cambia un ciclo a medias. Cada semáforo del sistema y cada buffer protegen su estado interno con un `Semaphore` de Java, porque procesos de computadores distintos (hilos distintos) pueden operarlos al mismo tiempo.

Bandeja de despertares. Cuando un proceso hace `semSignal` y despierta a otro que vive en otro computador, no toca las colas del otro computador (eso sería una carrera entre hilos): le deja el aviso en una bandeja protegida por un semáforo, y el otro computador la procesa al comienzo de su siguiente ciclo, como una interrupción.

Mapa de concurrencia: qué protege cada semáforo de Java. Hay dos niveles de semáforos que no deben confundirse: los `SemaforoSO` (`vacios`, `llenos`, `mutex`) son los semáforos *simulados* del sistema operativo y bloquean a los *procesos*; los `Semaphore` de Java de la tabla protegen las *estructuras de datos del simulador* entre los hilos.

Semáforo de Java	Qué protege o señala
`Cluster.cerrojo`	Todo el estado de la simulación (colas, CPU, RAM, procesos, buffers). El reloj lo mantiene tomado durante cada ciclo y la interfaz lo toma para leer o modificar
`RelojGlobal.candadoSuscriptores`	La lista de computadores suscritos al reloj
`RelojGlobal.pasos`	Señalización: un permiso es un ciclo pendiente cuando el reloj está en pausa

`Suscriptor.inicio` y `Suscriptor.fin`	Señalización: inicio del ciclo de un computador y barrera de fin de ciclo (el reloj espera a todos)
`Computador.candadoDespertares`	La bandeja de despertares de un computador (la escriben otros computadores)
`SemaforoSO.candado`	El valor y la cola de espera de un semáforo del SO
`BufferSO.candado`	Los contadores de elementos, producidos y consumidos de un buffer

Las colas de un computador (nuevos, listos, bloqueados y terminados) no necesitan semáforo propio: las toca solo el hilo de ese computador durante su ciclo, y la interfaz solo cuando tiene el cerrojo del cluster. El detalle completo (quién protege cada estructura, qué apunta cada puntero, un ciclo paso a paso y reglas para modificar el código) está en `docs/Guia_del_codigo.md`; además, todo el código fuente está documentado con Javadoc en cada clase, campo y método importante.

3. Enumeraciones

Enum	Valores	Para qué se usa
`EstadoProceso`	NUEVO, LISTO, EJECUCION, BLOQUEADO, TERMINADO	Estado de un proceso en su ciclo de vida
`TipoProceso`	CPU_BOUND, IO_BOUND, PRODUCTOR, CONSUMIDOR	Qué clase de proceso es
`ModoCPU`	USUARIO, SISTEMA_OPERATIVO	Si la CPU ejecuta un programa o trabajo del núcleo
`PoliticaPlanificacion`	FCFS, EDF, ROUND_ROBIN, PRIORIDADES	Política de cada computador (selector de la interfaz)
`ResultadoCiclo`	CONTINUA, BLOQUEADO, TERMINADO	Qué le pasó a un proceso tras ejecutar su instrucción
`TipoSemaforo`	CONTEO, BINARIO	Semáforo con varias unidades o mutex
`PasoComunicacion`	TRABAJAR, WAIT_PRIMERO, WAIT_MUTEX, ACCESO_BUFFER, SIGNAL_MUTEX, SIGNAL_SEGUNDO	Paso del protocolo productor-consumidor

Se usan enums y no constantes sueltas porque son conjuntos fijos de valores: el compilador impide valores inválidos y los `switch` quedan completos.

4. Interfaces

Interfaz	Para qué existe	Implementaciones
`PoliticaPlanificacionStrategy`	La política se puede cambiar en ejecución y agregar una nueva no obliga a modificar el planificador (patrón Strategy)	`FCFS`, `EDF`, `RoundRobin`, `PrioridadesApropiativas`
`Clockable`	Todo componente que reacciona al reloj comparte una interfaz: en cada ciclo el reloj llama a `tic` una vez	`Computador` (y a futuro un DMA)
`ProgramaProceso`	Cada tipo de proceso tiene su propio "código" y el computador no necesita saber cuál es: un tipo nuevo no obliga a tocar el núcleo	`ProgramaCpuBound`, `ProgramaIoBound`, `ProgramaProductorConsumidor`
`CriterioAsignacion`	El criterio de balanceo de carga es intercambiable sin tocar el `Cluster`	`MenorCarga`

5. Estructuras de datos propias y librerías

No se usa ninguna colección de `java.util` (`ArrayList`, `Queue`, `Stack`, `Vector`, `List`, `Map`, etc.). Las

estructuras son propias:

- `Nodo` y `ListaEnlazada`: almacenamiento genérico con recorrido por nodos.
- `Cola`: FIFO, usada para las colas de nuevos, listos, bloqueados y terminados y para las colas de espera de los semáforos.
- `Pila`: LIFO.

Las únicas partes de la biblioteca estándar usadas son Swing/AWT (interfaz gráfica), `java.util.concurrent.Semaphore` (exclusión mutua), `java.io` y `java.nio.file` (leer y escribir el CSV) y `java.util.function.Consumer` (una función de aviso, no una colección). El gráfico de utilización se dibuja con Java2D, sin librerías externas. Las estructuras que crecen (por ejemplo, el historial del gráfico) usan arreglos nativos.

6. Clases y métodos más importantes

`Proceso` (PCB). ID único global generado dinámicamente, nombre, computador, estado, tipo, prioridad, memoria asignada, deadline, tiempo restante y contador de programa (PC). Además guarda el motivo de bloqueo, el semáforo donde espera y los ciclos de llegada, primer despacho y fin (para las métricas). Métodos: `ejecutarUnCiclo()` (consume una instrucción y avanza su PC), `descontarDeadline()` y `cumplioDeadline()`.

`Computador`. Es la unidad reutilizable: agregar un computador es crear otra instancia. Tiene una `CPU`, una `RAM`, una política de planificación y las colas de nuevos, listos, bloqueados y terminados.

Método	Qué hace
`tic(ciclo)`	Un ciclo de reloj: procesa despertares, avanza los bloqueos por tiempo, despacha según la política, ejecuta una instrucción y vence los deadlines
`recibirProceso(p)` / `intentarAdmitirNuevos()`	Admisión de largo plazo: un proceso pasa de Nuevo a Listo solo si hay RAM suficiente
`terminarProceso(p)`	Libera su memoria, lo pasa a Terminados y revisa si algún proceso de Nuevos ya puede entrar
`bloquearPorSemaforo` / `bloquearPorTiempo`	Pasan el proceso a Bloqueado con su motivo y liberan la CPU
`liberarSemaforo(s)`	`semSignal` de un proceso; el proceso despertado se avisa por la bandeja de despertares
`setPolitica(estrategia, nombre)`	Cambia la política en ejecución, de forma independiente en cada computador

`Cluster`. Agrupa los computadores y los buffers, guarda la latencia de red, crea el reloj global (`crearReloj`), asigna procesos de forma manual o automática y entrega el log de eventos a la interfaz.

`SemaforoSO`. Semáforo propio: `semWait(proceso)` devuelve `false` si el proceso no puede continuar y queda en la cola del semáforo; `semSignal()` devuelve el proceso que debe despertarse. Es de conteo o binario. No bloquea hilos de Java: bloquea procesos.

`BufferSO`. Buffer acotado con los semáforos `vacios`, `llenos` y `mutex`. Ocupa RAM en su computador anfitrión (4 unidades por hueco).

Programas de proceso. `ProgramaCpuBound` ejecuta una instrucción de usuario por ciclo; `ProgramaIoBound` pide una E/S después de cada ráfaga de CPU y se bloquea mientras dura; `ProgramaProductorConsumidor` ejecuta el protocolo con semáforos.

Reloj global (`RelojGlobal`). `iniciar()`, `pausar()` y `paso()` (un solo ciclo); se puede cambiar su velocidad en ejecución.

7. Procesos y estados

El modelo de estados es: **Nuevo, Listo, Ejecución, Bloqueado y Terminado** (diagrama de la sección 13).

Estados de bloqueo y su justificación. Bloqueado no es un solo estado: un proceso bloqueado siempre tiene un motivo, que se muestra en la interfaz, y cada motivo termina por una causa distinta y la detecta un componente distinto:

Motivo	Cuándo ocurre	Cómo termina
Esperando semáforo	`semWait` sobre un semáforo sin unidades (buffer lleno, vacío o región crítica ocupada)	Otro proceso hace `semSignal`; el proceso despertado pasa a Listo
Esperando E/S	Un proceso I/O bound termina su ráfaga de CPU	Pasan los ciclos de la E/S
Esperando red	Acceso a un buffer que vive en otro computador	Pasan los ciclos de la latencia de red

Distinguirlos es necesario porque el de semáforo es indefinido (depende de otro proceso, y puede causar interbloqueo si se invierte el orden de los `semWait`), mientras que los otros dos son temporizados. Además, solo el bloqueo por semáforo cuenta para la métrica de tiempo bloqueado en semáforos.

Deadline. El deadline baja un ciclo por cada ciclo de reloj para todo proceso vivo (en CPU, listo, bloqueado o esperando memoria en Nuevos), contando desde su llegada. Al llegar a 0 el proceso se termina y libera todo lo que tuviera: su RAM, su lugar en la cola de un semáforo y, si estaba a mitad del protocolo, el mutex o el hueco que hubiera tomado, para no dejar un buffer atascado. Un proceso cumple su deadline si ejecutó todas sus instrucciones antes de que llegara a 0.

8. Planificación

Cada computador tiene su propia política, que se cambia en ejecución desde la interfaz sin afectar a los demás. El ordenamiento de la cola de listos está programado a mano.

Política	Cómo elige	¿Apropiativa?
FCFS	El primero en llegar	No
EDF	El deadline más cercano (se inserta ordenado por deadline)	No
Round Robin	Por turnos; el quantum es configurable en ejecución	Por quantum
Prioridades apropiativas	La mayor prioridad; si llega un listo de mayor prioridad le quita la CPU de inmediato	Sí

Asignación de procesos. El usuario elige el computador, o deja la opción "Automático". El criterio automático es **menor carga** (*join-the-shortest-queue*): el proceso va al computador con menos procesos vivos; si hay empate, al que tiene más RAM libre, y luego al de menor id; se descartan los computadores cuya RAM total nunca alcanzaría para el proceso. Se eligió porque no se conoce la duración de los procesos y no hay migración: mantener las colas parejas evita que un computador se sature mientras otro queda libre. Su limitación es que no distingue procesos largos de cortos. Una vez asignado, un proceso no cambia de computador.

9. Memoria principal

Cada computador tiene una RAM de tamaño configurable y cada proceso declara la memoria que requiere. Un proceso solo pasa de Nuevo a Listo si hay memoria suficiente; si no, espera en la cola de nuevos. Al terminar un proceso se libera su memoria y se revisa si algún proceso de Nuevos ya puede ser admitido. Los buffers también ocupan RAM en su computador anfitrión. No se usa asignación contigua ni particiones: se controla la memoria usada y la libre (el disco y la memoria virtual son el objetivo del Proyecto 2).

10. Sincronización: productor-consumidor

Cada buffer se sincroniza con el algoritmo clásico con tres semáforos:

```
productor:              consumidor:
  producir                 semWait(llenos)
  semWait(vacios)          semWait(mutex)
  semWait(mutex)           extraer
  insertar                 semSignal(mutex)
  semSignal(mutex)         semSignal(vacios)
  semSignal(llenos)        consumir
```

- El orden de los `semWait` importa: si el productor tomara primero el `mutex` y luego `vacios`, con el buffer lleno se quedaría con el mutex y nadie podría consumir (interbloqueo).

- Cada `semWait`, `semSignal`, inserción o extracción cuenta como una instrucción en **modo sistema operativo**; producir y consumir se ejecutan en modo usuario. Un productor o consumidor necesita `ciclos por elemento + 5` instrucciones por elemento.
- Soporta varios productores y consumidores por buffer, en el mismo computador del buffer o en otros.
- **Acceso remoto:** antes de cada acceso al buffer de otro computador, el proceso se bloquea tantos ciclos como la latencia de red (parámetro configurable), sin consumir instrucciones.
- Cuando un `semWait` no permite continuar, el proceso pasa a Bloqueado, entra a la cola del semáforo y la CPU atiende a otro proceso.

11. Interfaz gráfica, configuración y validaciones

- **Configuración inicial:** duración del ciclo, latencia de red, número de computadores (2 a 8) y, para cada uno, RAM, política inicial y quantum. Se puede cargar y guardar en un archivo.
- **Vista 1 (Crear proceso):** pide solo los campos del tipo elegido; el computador se elige o queda en "Automático".
- **Buffers:** formulario para crear buffers (nombre, capacidad, anfitrión) y tabla con elementos, capacidad, valores de los tres semáforos y procesos bloqueados en cada uno.
- **Vista 2 (un computador por pestaña):** proceso en la CPU con su PC, MAR, prioridad y deadline; si se ejecuta el SO o un programa de usuario; colas de nuevos, listos, bloqueados (con el motivo) y terminados con todos los elementos del PCB; ocupación de la RAM; selector de política y de quantum.
- **Global:** número de ciclo, botones iniciar, pausar y paso, duración del ciclo editable en ejecución y log de eventos de texto.
- **Métricas:** tablas por computador y del sistema, y un gráfico contra el tiempo con un selector para elegir la métrica: utilización de la CPU, throughput, tasa de éxito (deadlines cumplidos) o tiempo de espera. Hay una línea por computador y una punteada para el sistema completo.

Todos los campos validan tipo y rango (letras, decimales, vacíos, negativos y números gigantes se rechazan con un mensaje) sin interrumpir la simulación. El archivo de configuración es un CSV con líneas `CICLO_MS`, `LATENCIA_RED`, `COMPUTADOR`, `BUFFER` y `PROCESO`; al cargar un archivo inválido se indica el número de línea del error.

12. Métricas

Métrica	Definición
Throughput	Procesos completados por ciclo de reloj
Utilización	Ciclos en que la CPU ejecutó un proceso de usuario, entre los ciclos totales
Tiempo de respuesta promedio	Ciclos entre la llegada de un proceso y su primer despacho
Tiempo de espera promedio	Ciclos que cada proceso pasa en la cola de listos mientras la CPU ejecuta a otro
Cumplimiento de deadlines (tasa de éxito)	Procesos terminados a tiempo entre terminados
Equidad	Índice de Jain sobre la fracción de su tiempo en el sistema que cada proceso usó la CPU (1 = todos reciben lo mismo)
Elementos producidos y consumidos	Por buffer
Tiempo bloqueado en semáforos	Ciclos promedio que un productor o consumidor ha esperado en colas de semáforos

Las primeras seis se calculan por computador y para el sistema completo; el tiempo bloqueado en semáforos, también; los elementos producidos y consumidos, por buffer.

13. Diagramas UML

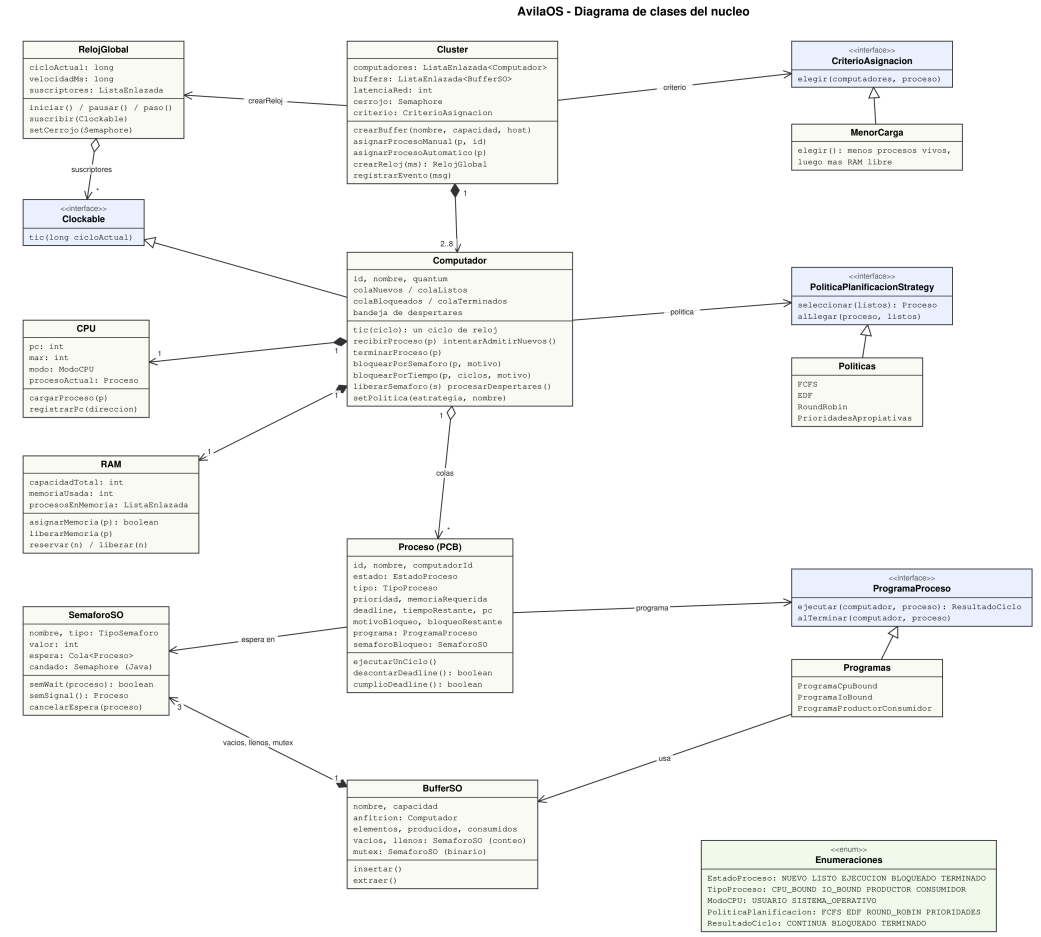


Diagrama de clases

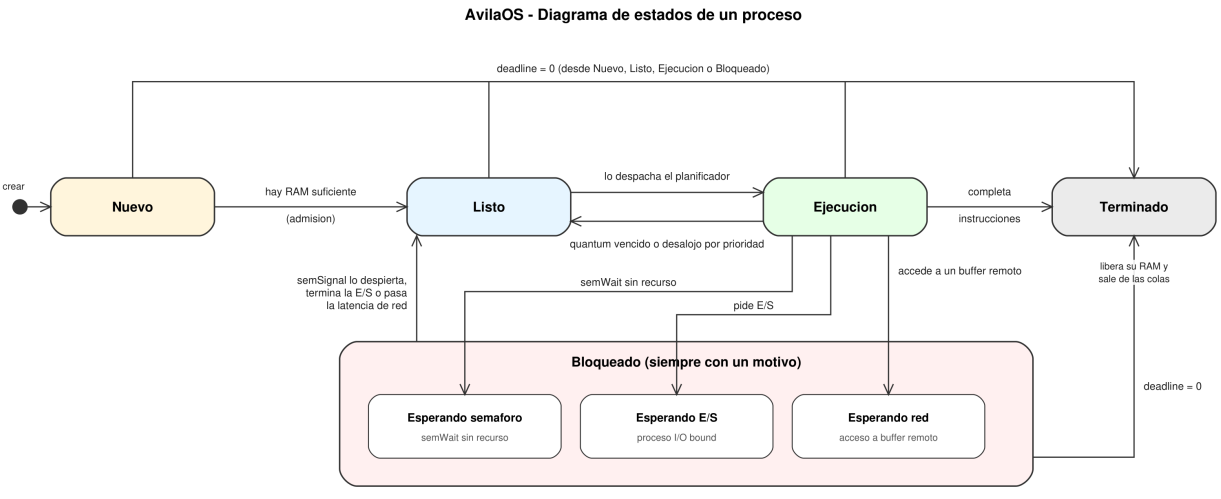


Diagrama de estados de un proceso

Estos son los diagramas del sistema terminado. Los diagramas que el equipo hizo en la fase de diseño, antes de implementar los semáforos, los buffers y el reloj con hilos, están en el Apéndice (sección 19).

14. Resultados y conclusiones

Los experimentos son deterministas (simulación ciclo a ciclo) y se pueden repetir con `java avilaos.pruebas.ExperimentoPoliticas`. Los resultados completos están en `docs/`

`resultados_experimentos.md`.

14.1 Comportamiento de cada política

Un computador, 8 procesos de CPU (instrucciones 8, 3, 6, 2, 5, 4, 7 y 1) que llegan entre los ciclos 0 y 5, con prioridades y deadlines distintos (algunos muy ajustados). Round Robin con quantum 3.

Política	Terminan todos	Throughput	Respuesta prom.	Cumplimiento	Equidad	Cambios de proceso
FCFS	ciclo 26	0.15	8.80	50 % (4 de 8)	0.38	0
EDF	ciclo 36	0.22	9.63	100 % (8 de 8)	0.75	0
Round Robin	ciclo 31	0.16	7.00	63 % (5 de 8)	0.65	5
Prioridades	ciclo 32	0.19	9.50	75 % (6 de 8)	0.65	2

Procesos que vencen su deadline: FCFS (P2, P4, P6, P8), EDF (ninguno), Round Robin (P4, P6, P8) y Prioridades (P2, P5).

- **FCFS** es la más simple pero la peor con deadlines: no mira la urgencia y un proceso corto y urgente espera detrás de uno largo. Cumple solo la mitad de los deadlines y tiene la equidad más baja.
- **EDF** cumple todos los deadlines porque esta carga es factible (el trabajo total son 36 instrucciones y todas caben a tiempo): atiende primero lo urgente. No da el mejor tiempo de respuesta, y termina en el ciclo 36 porque ningún proceso se pierde y todo el trabajo se ejecuta. Es la política con mejor throughput y la mejor equidad.
- **Round Robin** da el mejor tiempo de respuesta (7.00) porque todos reciben la CPU enseguida, pero no mira deadlines: deja morir a tres procesos urgentes.
- **Prioridades apropiativas** atiende primero lo importante pero deja morir a los de baja prioridad (P2 y P5) aunque su deadline fuera cercano: la prioridad y la urgencia no siempre coinciden.

Para esta carga con deadlines ajustados, EDF es la mejor opción. Para un sistema interactivo sin deadlines, Round Robin.

14.2 Efecto del quantum en Round Robin

Quantum	Respuesta prom.	Cumplimiento	Equidad	Cambios de proceso
1	3.14	50 %	0.84	25
2	5.14	50 %	0.83	11
3	7.00	63 %	0.65	5
5	7.80	63 %	0.61	3
8	8.80	50 %	0.38	0

Un quantum pequeño mejora la respuesta y la equidad pero multiplica los cambios de proceso (25 con quantum 1 frente a 5 con quantum 3). Un quantum grande hace que Round Robin se comporte como FCFS (con quantum 8 los resultados son idénticos a los de FCFS).

14.3 Productor y consumidor locales y remotos

Un productor y un consumidor de 12 elementos con un buffer de capacidad 2. En el caso local los dos están en el computador del buffer; en el remoto, el consumidor está en otro computador.

Caso	Latencia de red	Terminan en el ciclo	Bloqueo en semáforos (prom.)	Utilización del sistema
Local	—	144	20.50	50 %
Remoto	0	76	2.00	95 %
Remoto	2	112	18.00	70 %
Remoto	5	148	34.50	53 %
Remoto	10	208	62.00	38 %

- Con un solo computador, el productor y el consumidor se reparten una única CPU: tardan lo mismo que

la suma de su trabajo (144 ciclos).

- Repartirlos en dos computadores **los paraleliza** y casi reduce a la mitad el tiempo (76 ciclos con latencia 0).
- Cada acceso remoto cuesta la latencia de la red, así que a medida que crece, el beneficio desaparece: con latencia 5 el caso remoto ya es más lento que el local (148 frente a 144) y con 10 es mucho peor (208). Además aumenta el tiempo que los procesos pasan bloqueados en semáforos, porque el lado más lento hace esperar al otro.
- Conclusión: colocar productores y consumidores en computadores distintos conviene solo si la latencia es pequeña frente al trabajo por elemento; si no, es mejor mantenerlos junto al buffer.

14.4 Capacidad del buffer

Capacidad	Terminan en el ciclo	Bloqueo en semáforos (prom.)
1	144	44.50
2	144	20.50
4	144	8.50
8	144	4.50

Un buffer más grande no cambia el tiempo total cuando todo el trabajo es de CPU en una sola máquina, pero reduce mucho el tiempo que los procesos pasan bloqueados en semáforos (de 44.5 a 4.5 ciclos), porque absorbe las diferencias de ritmo entre productor y consumidor.

14.5 Contraste con las hipótesis iniciales del equipo

En la fase de diseño el equipo escribió qué esperaba de cada política y de los accesos remotos. Estas son esas hipótesis frente a lo que se midió:

Hipótesis del equipo	Resultado medido
FCFS: un proceso largo retrasa a otros con deadline cercano y el cumplimiento baja	Confirmada: cumple 50 % (4 de 8) y tiene la equidad más baja (0.38)
EDF mejora el porcentaje de procesos que terminan a tiempo frente a FCFS	Confirmada: 100 % frente a 50 % con esta carga. La parte de la hipótesis sobre deadlines parecidos no se midió
Round Robin mejora el tiempo de respuesta; con quantum muy pequeño aumenta el cambio de contexto y con uno muy grande se parece a FCFS	Confirmada: respuesta 7.00 frente a 8.80 de FCFS; 25 cambios con quantum 1 frente a 5 con quantum 3; con quantum 8 los resultados son idénticos a los de FCFS
Prioridades apropiativas: los procesos de baja prioridad pueden esperar demasiado (inanición)	Confirmada: P2 y P5 vencen su deadline esperando
Un deadline en 0 termina el proceso aunque le quede trabajo	Confirmada, también para procesos que esperan en listos, bloqueados o en la cola de un semáforo (tienen pruebas)
Al cambiar la política en caliente el computador no reinicia las colas, solo cambia el criterio de selección	Confirmada: una prueba cambia de FCFS a EDF con procesos ya en la cola y el siguiente en ejecutarse es el de deadline más cercano
Con acceso remoto los procesos pasan más tiempo bloqueados, baja la utilización de la CPU y el throughput es menor que en el caso local	Parcialmente confirmada: el bloqueo en semáforos y la caída de la utilización sí ocurren al crecer la latencia, pero el throughput solo es menor cuando la latencia es alta (con latencia 5 el caso remoto tarda 148 ciclos frente a 144 del local); con latencia baja el remoto termina antes porque usa dos CPU a la vez

15. Metodología de desarrollo, calidad y fiabilidad

Organización del trabajo. El proyecto se dividió en tres módulos para repartir el trabajo: el núcleo y la planificación; el hardware, el reloj y la sincronización; y la interfaz, la configuración, las métricas y el informe. Antes de programar, el equipo acordó los contratos del sistema (enumeraciones e interfaces) y documentó el diseño inicial con diagramas de PlantUML (Apéndice).

Control de versiones (GitFlow). El código está en un repositorio de GitHub con una rama `develop` y ramas por

funcionalidad con un objetivo claro (por ejemplo `feat/gui`, `feat/hardware`, `interfaz-pcb`, `metricas-y-grafica`, `sincronizacion-semaforos`, `nucleo-productor-consumidor`, `reloj-hilos`, `config-buffers-gui`, `pruebas-e-informe` y `grafica-metricas`). Cada rama se integra a `develop` con un Pull Request. Los commits son pequeños y con mensajes descriptivos con prefijo (`feat:`, `fix:`, `docs:`, `test:`, `refactor:`).

Desarrollo incremental. Se siguió el orden que sugiere el enunciado: primero un solo computador ejecutando procesos con sus políticas, luego los semáforos y el buffer local, después el acceso remoto con latencia y varios computadores, y al final las métricas, el gráfico y el informe. Cada paso se verificó antes de seguir.

Aseguramiento de la calidad.

- 363 pruebas automáticas (sin librerías externas) escritas junto con cada funcionalidad, con valores calculados a mano; incluyen 30 repeticiones de simulaciones con hilos reales para detectar carreras.
- Experimentos reproducibles (`ExperimentoPoliticas`) de donde salen las tablas del informe.
- El código compila con Java 21, no usa colecciones prohibidas (se verificó buscándolas) y está documentado con Javadoc en cada clase y método importante, con una guía de concurrencia (`docs/Guia_del_codigo.md`).

Seguridad y fiabilidad.

- **Exclusión mutua:** cada estructura compartida entre hilos tiene un dueño que la protege con un `Semaphore` (sección 2). El orden de los `semWait` evita el interbloqueo del productor-consumidor.
- **Sin recursos perdidos:** un proceso que muere por deadline a mitad del protocolo devuelve el mutex, el hueco o el elemento que tuviera tomado; un proceso bloqueado que muere sale de la cola del semáforo.
- **Entradas inválidas:** todos los campos validan tipo y rango sin interrumpir la simulación, y un archivo de configuración inválido se rechaza indicando la línea del error.
- **Fallos internos:** si un computador lanza una excepción durante un ciclo, el reloj se pausa y avisa al usuario en lugar de seguir con el estado a medias.
- **Casos borde probados:** buffers que no caben en la RAM, procesos más grandes que cualquier RAM, buffer de capacidad 1 con varios productores y consumidores, latencia 0 y alta, quantum de 1, y cambios de política y de quantum en ejecución.

16. Pruebas

Las pruebas automáticas (363 casos, sin librerías externas) se ejecutan con `avilaos.pruebas.EjecutarPruebas` (en NetBeans: clic derecho sobre el archivo y *Run File*). Cubren: semáforos (incluida una carrera de 4 hilos sobre el mismo semáforo), buffers, programas de proceso paso a paso, simulaciones completas, el reloj global (30 repeticiones con un hilo por computador para detectar carreras), el balanceo de carga, la configuración (guardar, cargar y validar), los formularios, las métricas y el gráfico (con valores calculados a mano) y las cuatro políticas.

17. Limitaciones y trabajo futuro

- EDF no es apropiativa: un proceso que llega con un deadline más cercano espera a que termine el que está en ejecución. Solo Prioridades desaloja.
- El criterio de balanceo no distingue procesos largos de cortos.
- Los nombres de computadores, procesos y buffers no pueden contener comas (por el formato CSV).
- Este proyecto se diseñó para extenderse en el Proyecto 2 (disco y memoria virtual, incluida la suspensión de procesos): el computador es una unidad reutilizable, las políticas, los programas de proceso y el criterio de asignación son interfaces, y la interfaz gráfica solo observa al sistema.

18. Cómo ejecutar

1. Abrir el proyecto en NetBeans (Java 21 o superior).
2. Ejecutar el proyecto (*Run*): la clase principal es `avilaos.gui.AvilaOSGui` y abre la ventana de configuración.
3. Pruebas: `avilaos.pruebas.EjecutarPruebas`. Experimentos:

`avilaos.pruebas.ExperimentoPoliticas`.

19. Apéndice: diseño inicial del equipo

Estos diagramas (hechos con PlantUML en la fase de diseño) fueron la base del proyecto. El sistema evolucionó al implementarlo: por ejemplo la interfaz `Ciclico` pasó a ser `Clockable`, los campos de RAM y de modo de la CPU pasaron a las clases `RAM` y `CPU`, `decrementarDeadline` pasó a `descontarDeadline` y se aplica a todos los procesos vivos, `admitirProceso` pasó a `recibirProceso` e `intentarAdmitirNuevos`, el estado Bloqueado se dividió en tres motivos y se agregaron los semáforos, los buffers, los programas de proceso y el reloj con hilos. Los diagramas del sistema terminado están en la sección 13.

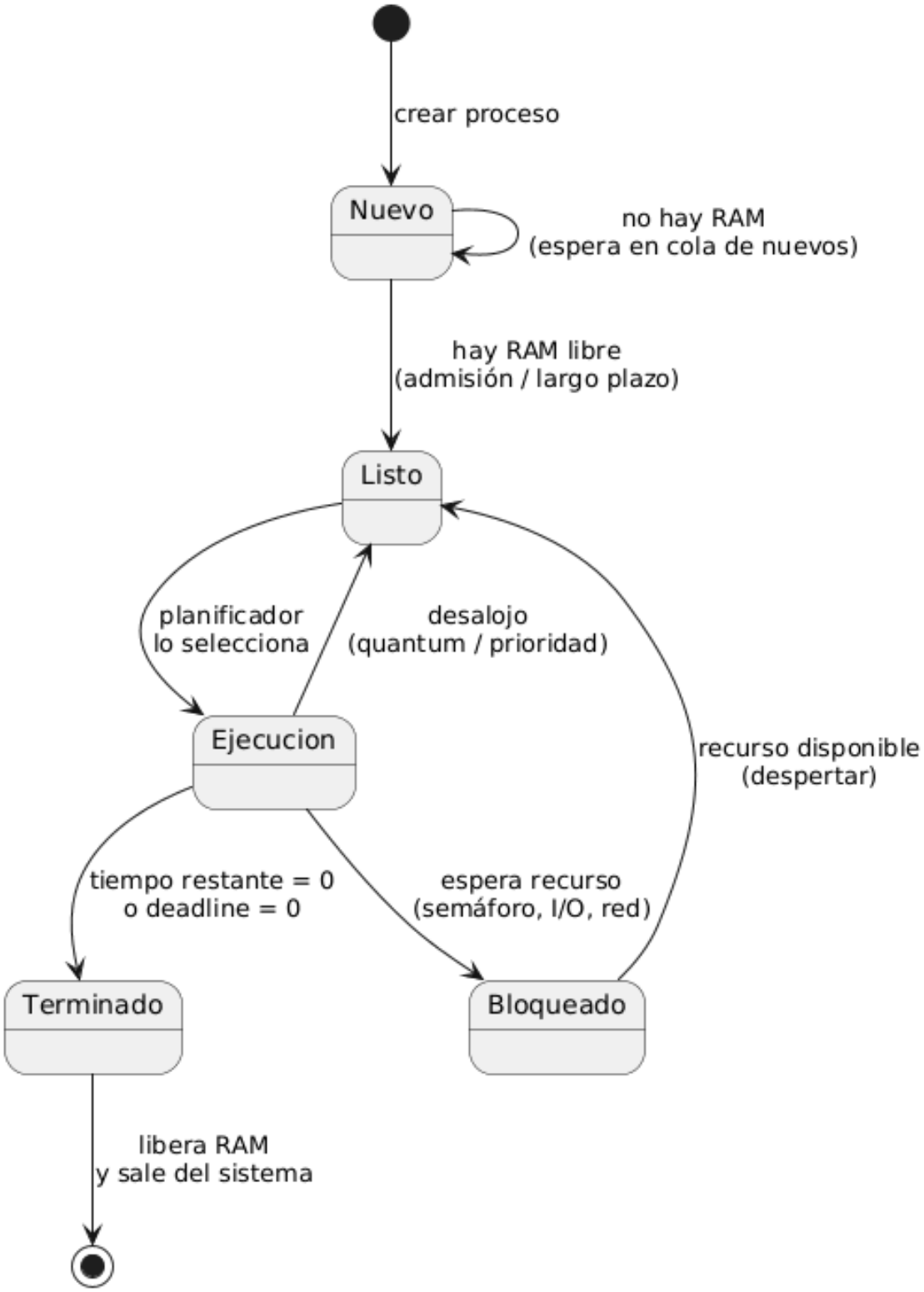


Diagrama de procesos inicial (PlantUML)

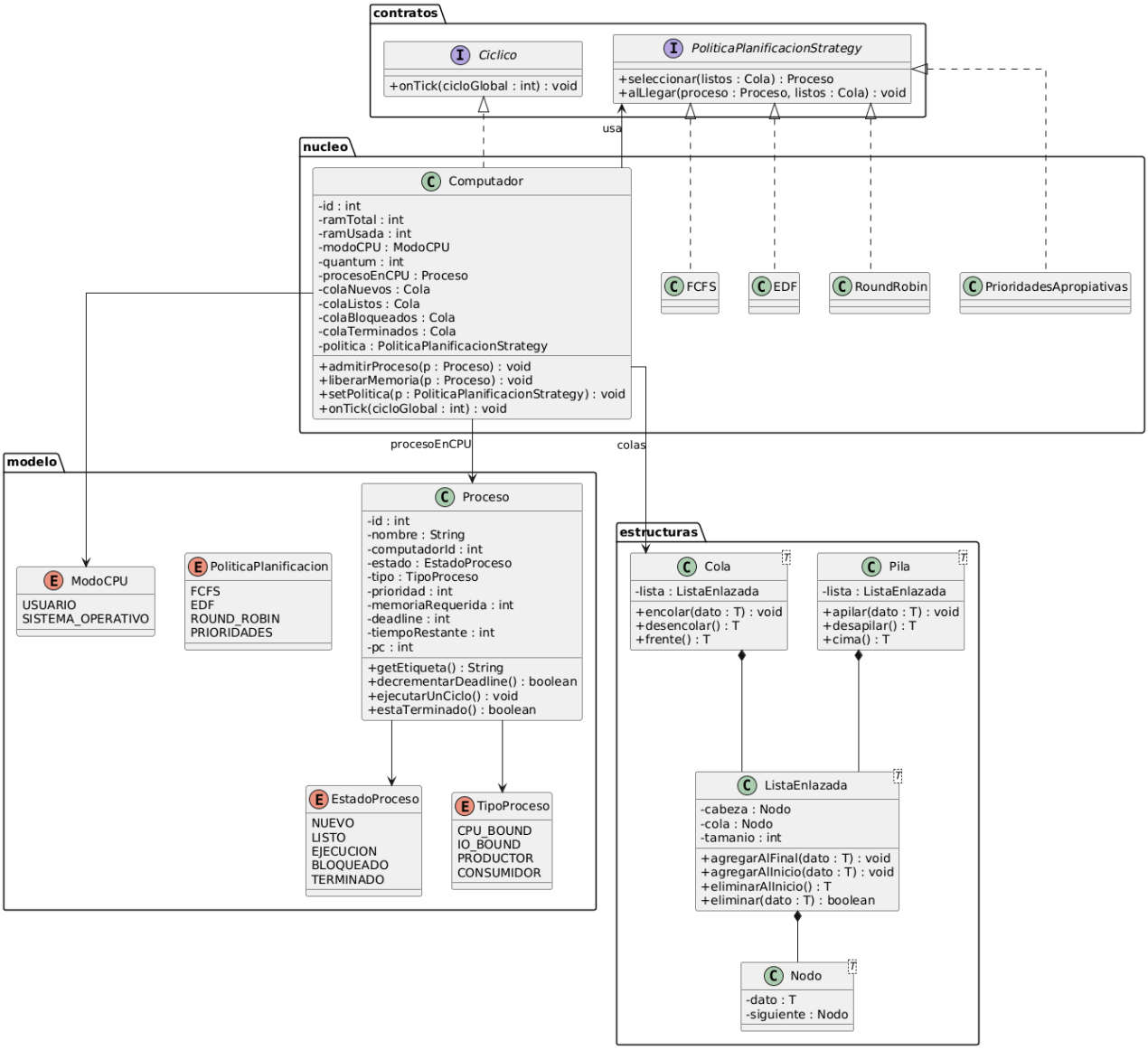


Diagrama de clases inicial (PlantUML)